BENEMÉRITA UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
Departamento de Ciencias Computacionales

Actividad de Aprendizaje 10. Programa 5. Algoritmo de Planificación RR (Round-Robin)

2026-B
Sección D09
Sistemas Operativos
Código de la materia: IL366

P R E S E N T A (N)
ICOM - Velázquez Vazquéz Valeria - 224785718

PROFESOR(A)
Becerra Velázquez Violeta del Rocío

Guadalajara, Jalisco, Mx.
27 de Octubre de 2026

Índice

Objetivo de la actividad..... 3
Notas acerca del lenguaje.....3
Arquitectura y estructuras de datos.....3
Reto principal y cómo se solucionó.....4
Capturas de pantalla.....5
Conclusiones.....6
Bibliografía..... 6

Tabla de Imágenes

Ilustración 1. Solicitud del número de procesos y del valor del Quantum.....5
Ilustración 2. Simulación en curso con la cola de listos como carrusel.....5
Ilustración 3. Tabla de procesos (BCP) desplegada con la tecla B.....6
Ilustración 4. Tabla final de procesos al terminar la simulación.....6

Objetivo de la Actividad

Modificar el simulador de la Actividad 8 para que la cola de listos se administre con el algoritmo de planificación Round-Robin (RR). El programa solicita al inicio el número de procesos y el valor del Quantum; cada proceso usa el procesador, como máximo, un Quantum de tiempo y, si no ha terminado, sale del procesador y se forma al final de la cola de listos, de modo que la cola funciona como un carrusel.

Se conservan todas las especificaciones del programa 4: modelo de cinco estados, máximo de cinco procesos en memoria, cola de bloqueados con espera de 8 unidades, procesos terminados, tabla de procesos (BCP) y las teclas "E" (interrupción por entrada/salida), "W" (error), "P" (pausa), "C" (continuar), "N" (nuevo proceso) y "B" (tabla de procesos). Además se muestra en pantalla el valor del Quantum y el tiempo transcurrido del Quantum del proceso en ejecución.

Notas acerca del Lenguaje

Se utilizó Python con la biblioteca gráfica PySide6 (Qt). El reloj sigue basándose en un QTimer que dispara un ciclo por segundo. Los datos iniciales se solicitan con dos llamadas consecutivas a QInputDialog.getInt: el número de procesos y el valor del Quantum, que debe ser mayor o igual a 1. Si el usuario cancela cualquiera de los dos cuadros, el programa termina sin abrir la simulación.

Para que el avance del carrusel sea visible, el primer renglón de la cola de listos, que es el siguiente proceso en recibir el procesador, se resalta con un color distinto mediante QColor. Cada vez que un proceso agota su Quantum y pasa al final de la cola, el resaltado cambia de proceso. La captura de teclas continúa siendo directa mediante keyPressEvent y una señal (Signal) hacia el controlador.

Arquitectura y Estructuras de Datos

El programa conserva la estructura Modelo-Vista-Controlador de la entrega anterior (main.py, config.py, models, views, controllers y utils), por lo que el cambio de FCFS a Round-Robin se concentró en pocos puntos:

Archivo	Cambio para Round-Robin
main.py	Pide también el valor del Quantum y lo pasa al motor.
config.py	Valor por defecto y límites del Quantum.
models/process.py	Nuevo campo quantum_elapsed: unidades usadas del Quantum actual.
models/simulator.py	Recibe el Quantum; _dispatch() asigna el CPU y _run_cpu() aplica la regla del Quantum.
views/main_window.py	Muestra el Quantum, su avance y resalta el siguiente de la cola de listos.
controllers/sim_controller.py	Sin cambios: el controlador no depende del algoritmo.

Tabla 1. Cambios respecto al programa 4

Las colas new_queue, ready_queue, blocked_queue y terminated_queue siguen siendo listas FIFO. El comportamiento circular de Round-Robin no necesita otra estructura: cuando el Quantum se agota, el proceso en ejecución se agrega al final de ready_queue con append() y el procesador se asigna al nuevo primer elemento con pop(0).

Reto Principal y Cómo se Solucionó

El principal reto fue definir el orden de los eventos dentro de cada unidad de tiempo para que el algoritmo fuera correcto en los casos límite. En cada ciclo se avanza el reloj, se actualiza la cola de bloqueados y se ejecuta una unidad del proceso actual; después se revisa primero si el proceso terminó y solo entonces si agotó el Quantum. Así, un proceso que termina justo en la última unidad de su Quantum se marca como terminado y no regresa a la cola de listos con tiempo restante cero.

Un segundo reto fue evitar unidades de tiempo vacías en el procesador. Al terminar un proceso, agotar su Quantum o presionar las teclas "E" o "W", se vuelve a asignar de inmediato el procesador al siguiente proceso listo y se admite un proceso nuevo si se liberó memoria. Con ello, cuando no hay bloqueos, el reloj final coincide con la suma de los TME. Al presionar "E" el contador del Quantum del proceso se reinicia, de modo que al regresar de bloqueados recibe un Quantum completo.

Para validar el algoritmo se agregaron pruebas automáticas del motor sin interfaz gráfica: ningún proceso usa el procesador más del Quantum de manera continua, el proceso que agota su Quantum queda al final de la cola de listos, un Quantum muy grande reproduce el comportamiento de FCFS, se respeta el máximo de cinco procesos en memoria y el tiempo de bloqueo es de 8 unidades. Además, el motor se comparó contra una versión previa del algoritmo con distintos valores del Quantum y secuencias de teclas, obteniendo los mismos resultados en cada unidad de tiempo.

Al ejecutar el programa para tomar las capturas se detectó un último detalle: al abrir la tabla de procesos con la tecla "B", la tabla interna del diálogo recibía el foco del teclado y la búsqueda

por teclado de Qt se quedaba con la letra "C", por lo que la ventana no se cerraba con esa tecla. Se corrigió quitándole el foco a la tabla con setFocusPolicy(Qt.NoFocus), de modo que la tecla llega al diálogo y la simulación continúa exactamente donde se quedó.

Capturas de Pantalla

Las siguientes ilustraciones corresponden a una ejecución del programa con 8 procesos iniciales y un Quantum de 3. Durante la simulación se usaron las teclas E (dos interrupciones de entrada/salida), W (el proceso 3 terminó por error), N (se generó el proceso 9), P y C para pausar y continuar, y B para consultar la tabla de procesos.

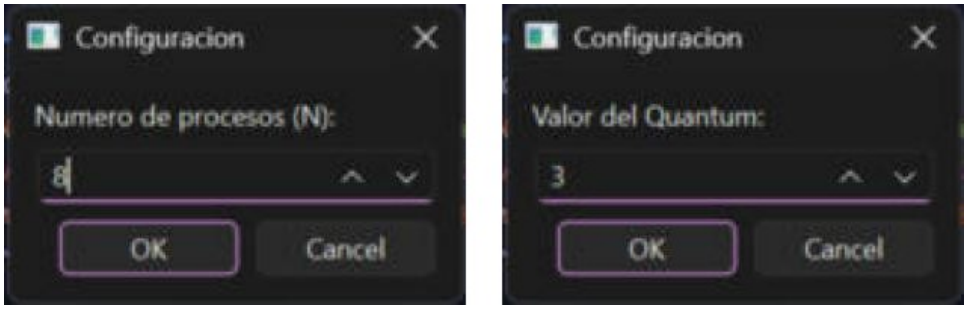


Ilustración 1. Solicitud del número de procesos y del valor del Quantum

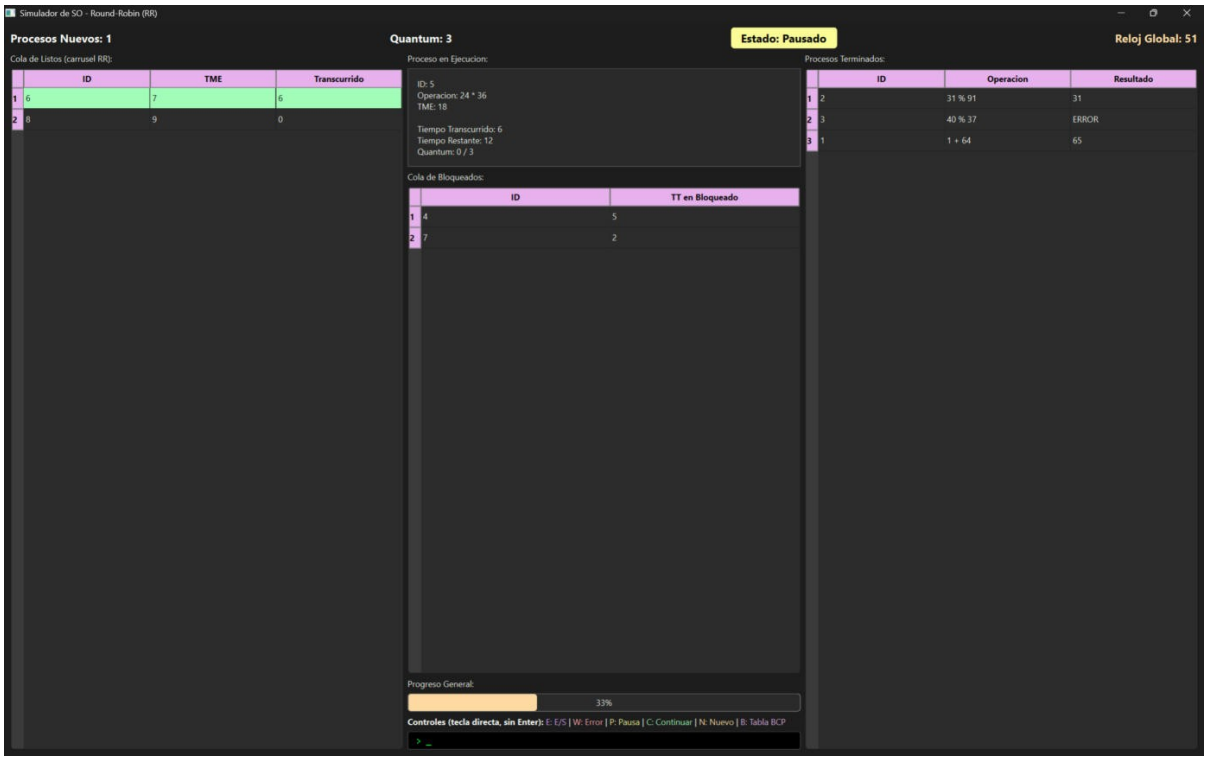
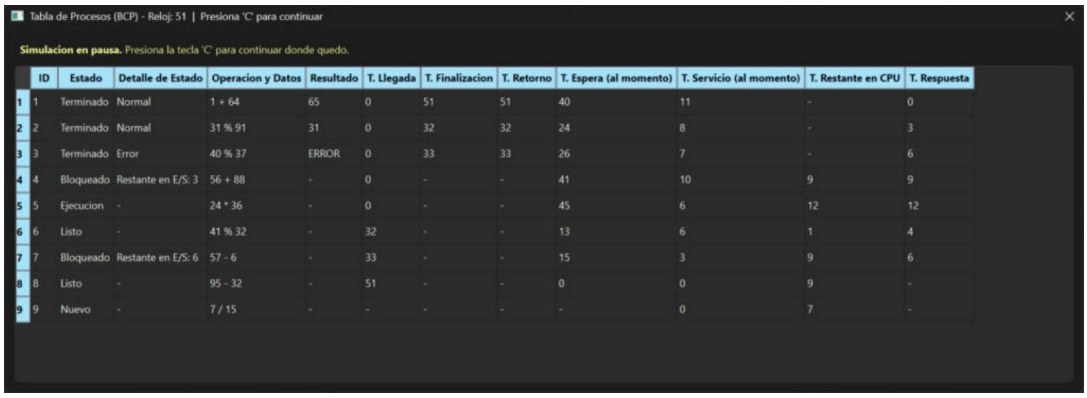
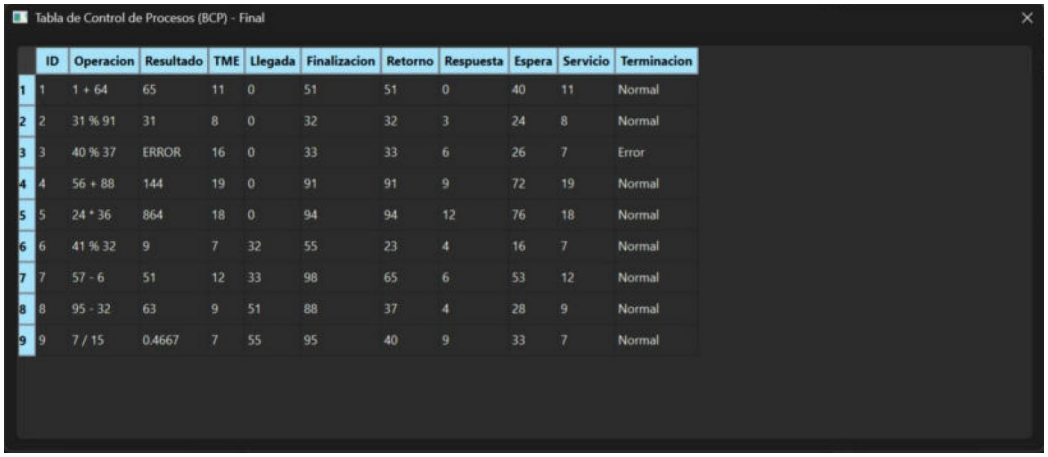


Ilustración 2. Simulación en curso con la cola de listos como carrusel



	ID	Estado	Detalle de Estado	Operacion y Datos	Resultado	T. Llegada	T. Finalizacion	T. Retorno	T. Espera (al momento)	T. Servicio (al momento)	T. Restante en CPU	T. Respuesta
1	1	Terminado	Normal	1 + 64	65	0	51	51	40	11	-	0
2	2	Terminado	Normal	31 % 91	31	0	32	32	24	8	-	3
3	3	Terminado	Error	40 % 37	ERROR	0	33	33	26	7	-	6
4	4	Bloqueado	Restante en E/S: 3	56 + 88	-	0	-	-	41	10	9	9
5	5	Ejecucion	-	24 * 36	-	0	-	-	45	6	12	12
6	6	Listo	-	41 % 32	-	32	-	-	13	6	1	4
7	7	Bloqueado	Restante en E/S: 6	57 - 6	-	33	-	-	15	3	9	6
8	8	Listo	-	95 - 32	-	51	-	-	0	0	9	-
9	9	Nuevo	-	7 / 15	-	-	-	-	-	0	7	-

Ilustración 3. Tabla de procesos (BCP) desplegada con la tecla B



	ID	Operacion	Resultado	TME	Llegada	Finalizacion	Retorno	Respuesta	Espera	Servicio	Terminacion
1	1	1 + 64	65	11	0	51	51	0	40	11	Normal
2	2	31 % 91	31	8	0	32	32	3	24	8	Normal
3	3	40 % 37	ERROR	16	0	33	33	6	26	7	Error
4	4	56 + 88	144	19	0	91	91	9	72	19	Normal
5	5	24 * 36	864	18	0	94	94	12	76	18	Normal
6	6	41 % 32	9	7	32	55	23	4	16	7	Normal
7	7	57 - 6	51	12	33	98	65	6	53	12	Normal
8	8	95 - 32	63	9	51	88	37	4	28	9	Normal
9	9	7 / 15	0.4667	7	55	95	40	9	33	7	Normal

Ilustración 4. Tabla final de procesos al terminar la simulación

Conclusiones

Pasar de FCFS a Round-Robin permitió observar cómo el Quantum cambia el reparto del procesador: los procesos cortos ya no esperan a que terminen los largos, lo que reduce el efecto convoy y mejora el tiempo de respuesta, a cambio de más cambios de contexto y de un mayor tiempo de retorno en los procesos largos. Un Quantum muy pequeño aumenta las rotaciones de la cola y uno muy grande hace que el comportamiento se parezca al de FCFS.

La separación en modelo, vista y controlador facilitó el cambio: el algoritmo se concentró en SimulatorEngine y Process, mientras que la ventana solo requirió mostrar el Quantum y su avance, y el controlador no tuvo que modificarse. Esto confirma la utilidad de mantener la lógica de planificación independiente de la presentación.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10a ed.). Wiley.

Stallings, W. (2018). Operating Systems: Internals and Design Principles (9a ed.). Pearson.

The Qt Company. (2024). PySide6 Reference Documentation — QMainWindow, QTimer, QInputDialog y QColor. Recuperado de la documentación oficial de Qt for Python.